Android, MySQL, PHP, & JSON 3 | Working wit Remote Server and MySQL

Posted By trav on May 29, 2013 | 0 comments

This entry is part 19 of 22 in the series Android-Intermediate

Configuring our MySQL Database and a PHP Environment!

In this Android tutorial we'll be setting up our remote MySQL database, testing out our php skills, and we'll start building our web service. There are two different sections for this tutorial. One is a walk-through guide of how to set everything up on your server if you have one, and the other is for setting up a local server on your computer with software such as XAMPP. If you have your own hosting package and domain, you are in the right place! **If you don't have hosting and want to use your computer to simulate an apache server as well as store your MySQL database, please check out the previous tutorial in this series.**

Follow the Remote Databases for Android Series!

- Part 1: Android Remote Databases and Servers Overview
- Part 2: Local Xampp Server and MySQL Database
- **Part 3: Working with a Remote Server and MySQL**
- Part 4: Creating Login and Registration PHP Scripts
- Part 5: Developing the Android Application
- Part 6: JSON Parsing and Android ListView Design

Remote

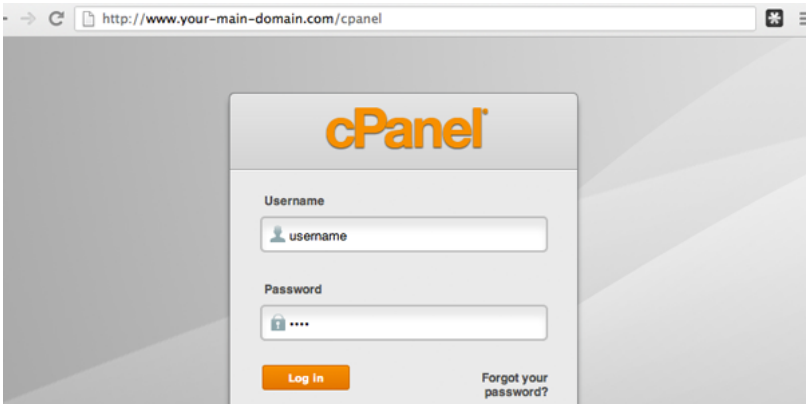
Database Setup for Android Video Tutorial

Setting up our Remote

Database for Android

Step 1: Setting Up Our Database

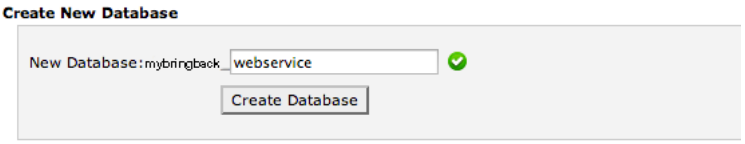
The first thing we need to to is set up a place to store all our user information. Login into your cPanel, usually it is just your main domain name forward-slash cpanel :



Once you are logged in, look for the database section of cPanel. It should contain MySQL Databases, MySQL Database Wizard, phpMyAdmin, and Remote MySQL and look something like this:

Let's setup our database and add a couple of tables. Click on the "MySQL Databases" icon and create a new database called "webservice". You're hosting plan will have assigned you a username, such as mybringback. Therefore, our actual database will be "yourusername_webservice", or in my case "mybringback_webservice". Make sure you write this down, or remember it for later

when we work with PHP. Below is a screenshot of how to create a new MySQL Database. Once you click "Create Database", you should get a confirmation message, such as, "Added the Database mybringback_webservice".



Step 2: Adding a Database User

Cool, now we have a database to work with, but the problem is, no one is allowed to modify it. Therefore, we need to add a MySQL user to maintain our database, we will give this MySQL user all privileges, meaning they can add stuff, delete stuff, modify data, change table names, change column names, and do whatever else they feel like.

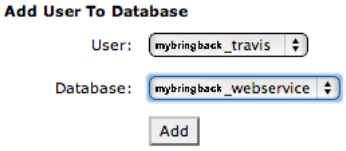
****Important Note- Even though our database will have a lot of user data within our tables, there will be ONLY ONE MySQL user. A MySQL user has access to the database information, where as, all other users are just data stored within the database. Don't get confused.****

Go back to the page where we created our database if you haven't already done so, and scroll down to where it says "Add New User". Let's fill out the needed information. I'm going to create a MySQL user named "travis", and you want to make sure you have a strong password because anyone that hacks into this username can manipulate our database however they would like. I would recommend using the "Password Generator" and even adding on a few extra characters and asterisks, but I'm just going to use the password "bacon1" for this username. Make sure you copy and paste your password somewhere secure, because we will need it later, and then click "Create User".

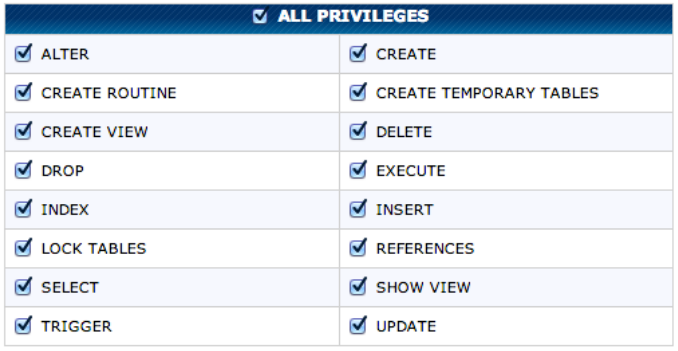
****Again, note that our username is "mybringback_travis" not just "travis".****

Step 3: Adding a MySQL User to a Database with All Privileges

This will be pretty simple, just go back to the page where we created our database and user and scroll to the bottom where it says "Add User To Database". Just select your newly created MySQL user and database and click "Add".



Next, select "ALL PRIVILEGES".



Click make changes, and we are done with the "MySQL Database" section of cPanel. Go ahead and close it and go back to the database section of cPanel and open up "phpMyAdmin". We are going to add some tables and columns to our database.

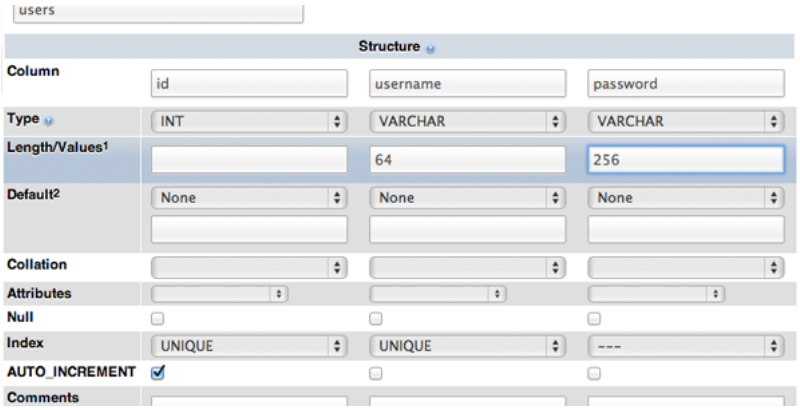
Step 4: Adding MySQL Tables and Field Names

Once you open up your phpMyAdmin, you can go ahead and click on our new added database ("_webservice"). After you do, you can easily create some new tables by either using the "Create table" button within the left sidebar, or under the "Structure" tab. Let's create a new table called 'users' and for now it will only have three columns: username, password, and id.

****Note – The standard syntax of MySQL database names should all be lowercase and contain no underscores (_). However, the standard syntax for a MySQL table name should include underscores and also be all lowercase. ****

The columns are called field names, these should be lowercase as well. We also don't want more than one user having the same username, and therefore we will tell our database we want the username to be unique. We can also specify the type for each field name, or column. We probably would want to use something else beside varchar for our username and password, but it will do for now. The "Length/Values" attribute defines the maximum number of characters for that field.

Lastly, we want to have the "id" field name to be unique and auto increment, this means the first user created will have an id of 0, and the next user will be 1, then 2, etc. Even if we had 500 users, deleted them all from our database and created a new user, they would have the id of 501, not 0 again. Don't worry if you're wondering why an id is useful, it's not too important right now. After you have set up your table like the image below, click "save".



Now, let's create a new table:

Name the table "comments".
Add four field names: post_id, username, title, message.
post_id will be of type int with value of 11, it will be unique, and auto increment.
The rest will be of type VARCHAR.
username length will be 64.
title length will be 120.
comment length will be 255.
We could set this up the same way, but I'll show you how we can make a simple SQL query to create the table for us. At the top of the page you should see "localhost > mybringback_webservice", click on the "mybringback_webservice" then click on the "SQL" table. Here we can write some SQL code, for example:

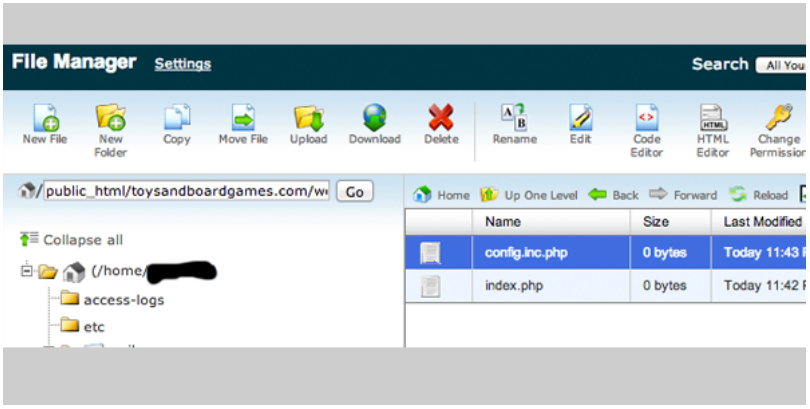
```
CREATE TABLE comments
(
post_id int(11) UNIQUE AUTO_INCREMENT,
username varchar(64),
title varchar(120),
message varchar(255)
)
```

Then just click GO, and our new table will be created.

That about covers it for setting up our MySQL database, tables, and field names for our Android application. Make sure you have your MySQL database name, MySQL privileged user name, and privileged user's password handy, we will need them in just a moment.

Step 5: Creating a Web Service Directory

Now that we have our database information set up, we need to start creating a PHP based web service. Back in cPanel, look for "File Manager" and open up the Web Root. Most likely you will just want to create a new folder called "webservice". This location will be your domain name then webservice (http://www.mybringback.com/webservice). I'm going to be using the domain toysandboardgames.com/webservice for this mini series. Mainly, because I purchased this domain a while back and I don't have anything on it. Within your "webservice" folder, create two new files, **"index.php"** and **"config.inc.php"**.



Edit you index.php file to simply echo a test message:

```php
<?php
echo "Testing, this should be visible at http://www.toysandboardgames.com/webservice/";
?>
```

Save the file and test the url to make sure you're seeing a message.

Now, let's setup our config file. This file will hold all database information that we will need to make changes to our database. Any time we include this file we can interact with our database using the defined variables. You'll see how this works later if you're confused, don't worry.

config.inc.php:
This modified script is from an excellent commented php file on building a simple and secure login system, and I just wanted to give credit where credit is due.

```php
<?php

    // These variables define the connection information for your MySQL database
    $host = "localhost";
    $dbname = "mybringback_webservice";
    $username = "mybringback_travis";
    $password = "bacon1";



    // UTF-8 is a character encoding scheme that allows you to conveniently store
    // a wide varienty of special characters, like ¢ or €, in your database.
    // By passing the following $options array to the database connection code we
    // are telling the MySQL server that we want to communicate with it using UTF-8
    // See Wikipedia for more information on UTF-8:
    // http://en.wikipedia.org/wiki/UTF-8
    $options = array(PDO::MYSQL_ATTR_INIT_COMMAND => 'SET NAMES utf8');

    // A try/catch statement is a common method of error handling in object oriented code.
    // First, PHP executes the code within the try block.  If at any time it encounters an
    // error while executing that code, it stops immediately and jumps down to the
    // catch block.  For more detailed information on exceptions and try/catch blocks:
    // http://us2.php.net/manual/en/language.exceptions.php
    try
    {
        // This statement opens a connection to your database using the PDO library
        // PDO is designed to provide a flexible interface between PHP and many
        // different types of database servers.  For more information on PDO:
        // http://us2.php.net/manual/en/class.pdo.php
        $db = new PDO("mysql:host={$host};dbname={$dbname};charset=utf8", $username, $password,
$options);
    }
    catch(PDOException $ex)
    {
        // If an error occurs while opening a connection to your database, it will
        // be trapped here.  The script will output an error and stop executing.
        // Note: On a production website, you should not output $ex->getMessage().
        // It may provide an attacker with helpful information about your code
        // (like your database username and password).
        die("Failed to connect to the database: " . $ex->getMessage());
    }

    // This statement configures PDO to throw an exception when it encounters
    // an error.  This allows us to use try/catch blocks to trap database errors.
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // This statement configures PDO to return database rows from your database using an
associative
    // array.  This means the array will have string indexes, where the string value
    // represents the name of the column in your database.
    $db->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);

```

```php
    // This block of code is used to undo magic quotes.  Magic quotes are a terrible
    // feature that was removed from PHP as of PHP 5.4.  However, older installations
    // of PHP may still have magic quotes enabled and this code is necessary to
    // prevent them from causing problems.  For more information on magic quotes:
    // http://php.net/manual/en/security.magicquotes.php
    if(function_exists('get_magic_quotes_gpc') && get_magic_quotes_gpc())
    {
        function undo_magic_quotes_gpc(&$array)
        {
            foreach($array as &$value)
            {
                if(is_array($value))
                {
                    undo_magic_quotes_gpc($value);
                }
                else
                {
                    $value = stripslashes($value);
                }
            }
        }

        undo_magic_quotes_gpc($_POST);
        undo_magic_quotes_gpc($_GET);
        undo_magic_quotes_gpc($_COOKIE);
    }

    // This tells the web browser that your content is encoded using UTF-8
    // and that it should submit content back to you using UTF-8
    header('Content-Type: text/html; charset=utf-8');

    // This initializes a session.  Sessions are used to store information about
    // a visitor from one web page visit to the next.  Unlike a cookie, the information is
    // stored on the server-side and cannot be modified by the visitor.  However,
    // note that in most cases sessions do still use cookies and require the visitor
    // to have cookies enabled.  For more information about sessions:
    // http://us.php.net/manual/en/book.session.php
    session_start();

    // Note that it is a good practice to NOT end your PHP files with a closing PHP tag.
    // This prevents trailing newlines on the file from being included in your output,
    // which can cause problems with redirecting users.



?>
```

Perfect, now we have the bare basics set up for our web service even though it doesn't do anything yet.

Previous Lesson Donate Next Lesson

Author: trav

I'm just an average guy that love programming.

CAPTCHA Image

CAPTCHA Code *

Comment

You may use these HTML tags and attributes: <a href="" title=""> <abbr title=""> <acronym title=""> <b> <blockquote cite=""> <cite> <code> <del datetime=""> <i> <q cite=""> <strike>

Submit Comment

Tutorials